1.1.2 — Tríada CIA y Hexad de Parker

Pregunta de inicio: Si una aplicación protege la contraseña del usuario, pero permite modificar sus notas o permanece caída durante horas, ¿podemos decir que es segura?

La seguridad no se reduce a "ocultar contraseñas". El modelo clásico **CIA** permite analizar tres propiedades fundamentales: **Confidencialidad, Integridad y Disponibilidad**.

Confidencialidad

La **confidencialidad** busca que la información sea accesible únicamente para personas, procesos o sistemas autorizados.

Ejemplos

- Un estudiante solo puede consultar sus propias calificaciones.
- Un empleado no puede acceder a expedientes del área de Recursos Humanos si no tiene autorización.
- Una API no entrega datos privados sin comprobar la identidad y permisos del solicitante.

Caso de software

Una aplicación usa /api/notas/1001. El estudiante cambia el ID por 1002 y obtiene datos de otra persona. La aplicación sigue funcionando, pero la confidencialidad está comprometida.

Controles asociados

- autorización por recurso;
- control de acceso;
- cifrado;
- manejo adecuado de sesiones;
- minimización de datos.

Integridad

La **integridad** busca que la información y los sistemas mantengan exactitud, consistencia y ausencia de modificaciones no autorizadas.

Ejemplos

- Una nota no puede modificarse sin autorización del docente.
- Un saldo bancario no debe cambiar por manipulación externa.
- Un pedido no debe cambiar de cantidad o precio sin registrar la operación.

Caso de software

Si la API acepta directamente saldo=99999 enviado por el cliente sin validación de autorización ni reglas de negocio, la integridad está en riesgo.

Controles asociados

- validación del lado servidor;
- reglas de negocio;
- controles de autorización;
- transacciones;
- firmas o verificaciones de integridad cuando corresponda.

Disponibilidad

La **disponibilidad** busca que los sistemas, servicios y datos estén accesibles cuando los usuarios autorizados los necesiten.

Ejemplos

- Portal académico disponible durante matrículas.
- API accesible durante una operación de pago.
- Base de datos operativa para consultas críticas.

Situaciones que afectan disponibilidad

- caída del servidor;
- saturación;
- errores de programación;
- dependencia externa caída;
- consumo excesivo de recursos;
- pérdida de almacenamiento.

CIA no son tres cajas aisladas

Los tres principios se relacionan entre sí.

Confidencialidad + Integridad

Un expediente médico no solo debe mantenerse privado; también debe mantenerse correcto.

Ejemplo: si un atacante puede leer un diagnóstico, se afecta confidencialidad. Si además puede modificarlo, se afecta integridad.

Integridad + Disponibilidad

Que un servicio esté “arriba” no sirve si entrega datos incorrectos.

Ejemplo: un sistema bancario responde correctamente, pero muestra saldos alterados. Hay disponibilidad, pero no integridad.

Confidencialidad + Disponibilidad

Un servicio debe estar disponible **para quien corresponde**, no para cualquier persona.

Ejemplo: una API académica puede estar disponible 24/7, pero debe restringir datos privados a usuarios autorizados.

CIA en conjunto

flowchart TD

C["Confidencialidad
¿Quién puede acceder?"] --- I["Integridad
¿La información es correcta?"]

I --- A["Disponibilidad
¿Está accesible cuando se necesita?"]

A --- C

¿Cómo puede un error de programación afectar CIA?

Error	**Propiedad afectada**	**Ejemplo**
Falta de autorización	Confidencialidad	Usuario A consulta información de B
Validación insuficiente	Integridad	Cliente modifica un valor crítico

Error	Propiedad afectada	Ejemplo
Bucle o consumo excesivo	Disponibilidad	Aplicación agota CPU
Permisos mal diseñados	C + I	Usuario lee y modifica datos ajenos
Error severo de lógica	C + I + A	Exposición, alteración y caída del sistema

Limitaciones de la tríada CIA

CIA es un modelo fundamental, pero no describe todas las propiedades que pueden ser importantes. Por ejemplo, saber que un archivo está íntegro no garantiza que sepamos quién lo creó realmente. Tampoco que un archivo esté disponible significa que sea útil.

Hexad de Parker

El **Parkerian Hexad** amplía el análisis clásico con tres propiedades adicionales:

- **Posesión o control**
- **Autenticidad**
- **Utilidad**

Junto con CIA forman seis propiedades:

flowchart LR

C["Confidencialidad"] --- P["Posesión / Control"]

I["Integridad"] --- AU["Autenticidad"]

A["Disponibilidad"] --- U["Utilidad"]

Posesión o control

Se refiere a quién tiene control físico o lógico sobre la información o el activo.

Ejemplo

Un portátil corporativo cifrado es robado. Tal vez el ladrón no pueda leer los datos, por lo que la confidencialidad puede mantenerse. Sin embargo, la organización perdió **posesión/control** del dispositivo.

Autenticidad

Busca establecer que una entidad, mensaje, archivo o transacción es realmente quien o lo que afirma ser.

Ejemplo

Un documento puede estar íntegro, pero si no sabemos quién lo emitió, la integridad por sí sola no garantiza autenticidad.

Utilidad

Se refiere a que la información esté en una forma que permita utilizarla para el propósito esperado.

Ejemplo

Un respaldo existe y está disponible, pero está en un formato incompatible o no puede restaurarse. Existe disponibilidad, pero no utilidad.

Comparación CIA vs Hexad de Parker

Modelo	Propiedades	Pregunta que ayuda a responder
CIA	Confidencialidad, Integridad, Disponibilidad	¿Quién accede? ¿Está correcto? ¿Está disponible?
Hexad	CIA + Posesión/Control + Autenticidad + Utilidad	Además: ¿quién lo controla? ¿es genuino? ¿sirve realmente?

Caso integrador

Una empresa posee una copia de seguridad cifrada en un disco externo. El disco es robado, pero el atacante no conoce la clave. Meses después, la empresa descubre que el formato del respaldo ya no puede restaurarse.

- **Confidencialidad:** posiblemente preservada por el cifrado.
- **Posesión/control:** perdida por el robo.
- **Disponibilidad:** la copia existía, pero ya no está bajo control de la organización.
- **Utilidad:** comprometida porque no puede restaurarse.

Miniquiz

1. Un usuario consulta datos de otro usuario sin autorización. ¿Qué propiedad se afecta principalmente?
2. Un pedido cambia de USD 50 a USD 500 sin autorización. ¿Qué propiedad se afecta?
3. El servicio está caído durante la matrícula. ¿Qué propiedad se afecta?
4. Un dispositivo cifrado es robado. ¿Qué propiedad adicional del Hexad ayuda a analizar el caso?
5. Un respaldo existe pero no puede restaurarse. ¿Qué propiedad adicional permite describir mejor el problema?

Resumen

1. CIA analiza confidencialidad, integridad y disponibilidad.
2. Las tres propiedades se relacionan y pueden verse afectadas simultáneamente.
3. Un error de programación puede comprometer una o varias propiedades.
4. El Hexad de Parker añade posesión/control, autenticidad y utilidad.
5. Modelos ampliados permiten describir situaciones que CIA por sí sola no explica completamente.

1.1.3 — Autenticidad, trazabilidad, responsabilidad y no repudio

Autenticidad → Trazabilidad → Responsabilidad → No repudio

Pregunta de inicio: si una acción aparece en un sistema, ¿podemos demostrar quién la realizó, cuándo ocurrió y evitar que luego se niegue esa acción?

Autenticidad

La **autenticidad** permite establecer que una identidad, entidad, mensaje, archivo o transacción es genuina y corresponde realmente a quien afirma ser.

Ejemplo de software

Un usuario inicia sesión con sus credenciales y un segundo factor. El sistema no solo recibe un nombre de usuario: intenta comprobar que la persona que se presenta como ljaramillo es realmente quien dice ser.

Riesgo

Si la autenticidad falla, un atacante puede hacerse pasar por otra persona, servicio o sistema.

Trazabilidad

La **trazabilidad** permite registrar, reconstruir y seguir las acciones realizadas dentro de un sistema.

Un log útil debería incluir

- fecha y hora;
- usuario o identidad;
- dirección IP o contexto de origen;
- operación realizada;
- recurso afectado;
- resultado;
- identificador de correlación cuando corresponda.

Log malo

ERROR LOGIN

Log útil

2026-09-30T11:20:45 user=estudiante17 ip=192.168.10.25 action=LOGIN result=FAILED reason=INVALID_CREDENTIALS

Por qué importa

Sin trazabilidad, una organización puede saber que “algo pasó”, pero no reconstruir cómo ocurrió.

Responsabilidad o accountability

La **responsabilidad** implica que las acciones realizadas puedan atribuirse a una entidad identificable y que exista claridad sobre quién debe responder por ellas.

Ejemplo

Si un administrador elimina registros, el sistema debe poder asociar esa acción con una cuenta individual y no con una cuenta genérica compartida como admin.

Mala práctica

Compartir una misma cuenta entre varias personas destruye gran parte de la capacidad de atribución.

No repudio

El **no repudio** busca disponer de evidencia suficiente para que una entidad no pueda negar posteriormente haber realizado una acción relevante.

Ejemplos

- firma digital de un documento;
- aprobación registrada de una transferencia;
- confirmación de una operación con evidencia verificable;
- transacción asociada a identidad, tiempo y mecanismos criptográficos.

Importante

El no repudio **no es sinónimo** de autenticidad o trazabilidad. Se apoya en ellas, pero añade la idea de evidencia que dificulta negar una acción realizada.

Relación entre los cuatro principios

flowchart LR

A["Autenticidad
¿Quién es?"] --> T["Trazabilidad
¿Qué hizo, cuándo y desde dónde?"] --> R["Responsabilidad
¿A quién se atribuye?"] --> N["No repudio
¿Puede negar la acción?"]

Caso integrador

Un docente modifica una calificación:

- **Autenticidad:** el sistema verifica la identidad del docente.
- **Trazabilidad:** registra fecha, usuario, operación y resultado.
- **Responsabilidad:** la modificación queda atribuida a una cuenta individual.

- **No repudio:** una evidencia adecuada puede demostrar que la acción fue aprobada y realizada por esa identidad.

Errores frecuentes

1. Pensar que autenticación y autenticidad son exactamente lo mismo.
2. Creer que guardar cualquier log garantiza trazabilidad.
3. Usar cuentas compartidas y luego pretender atribuir responsabilidades.
4. Confundir no repudio con “tener un historial”.

Microactividad

Relaciona cada situación con el principio principal:

1. El sistema confirma que un certificado pertenece realmente al servidor esperado.
2. Un log permite reconstruir quién cambió una nota y a qué hora.
3. Una acción queda atribuida a una cuenta individual identificable.
4. Existe evidencia verificable de que una persona aprobó una transacción.

1.1.4 — Amenaza, vulnerabilidad, riesgo y ataque

Pregunta de inicio: password = "admin123" ¿es una amenaza, una vulnerabilidad, un ataque o un incidente?

Modelo de análisis

ACTIVO → AMENAZA → VULNERABILIDAD → ATAQUE → IMPACTO → RIESGO → CONTROL → VERIFICACIÓN

flowchart LR

A["Activo"] --> B["Amenaza"] --> C["Vulnerabilidad"] --> D["Ataque"] --> E["Impacto"] --> F["Riesgo"] --> G["Control"] --> H["Verificación"]

Activo

Es aquello que tiene valor y debe protegerse.

Ejemplos

Credenciales, datos personales, código fuente, disponibilidad de una aplicación, reputación, secretos de API.

Amenaza

Una **amenaza** es una circunstancia, actor o evento con capacidad de causar daño a un activo.

Ejemplos

- ciberdelincuente;
- error humano;
- fallo eléctrico;
- malware;
- incendio;
- usuario interno malintencionado.

Idea clave

La amenaza **puede existir aunque todavía no haya un ataque**.

Vulnerabilidad

Una **vulnerabilidad** es una debilidad que puede ser aprovechada por una amenaza.

Ejemplos de software

- contraseña débil;
- secreto hardcodeado;
- API sin autorización;
- validación insuficiente;
- dependencia desactualizada;
- permisos excesivos.

Ataque

Un **ataque** es la acción concreta mediante la cual una amenaza intenta aprovechar una vulnerabilidad o comprometer un activo.

Ejemplo

Una contraseña débil es una vulnerabilidad. Probar sistemáticamente contraseñas para acceder a la cuenta es el ataque.

Impacto

El **impacto** representa las consecuencias que se producen cuando una amenaza logra materializarse.

Ejemplos

- fuga de datos;
- pérdida económica;
- caída del servicio;
- alteración de notas;
- sanciones;
- daño reputacional.

Riesgo

El **riesgo** expresa la posibilidad de que una amenaza aproveche una vulnerabilidad y produzca un impacto sobre un activo.

Forma conceptual

Riesgo ≈ probabilidad × impacto

No siempre es necesario calcularlo numéricamente en una primera aproximación, pero sí comprender qué puede ocurrir y qué tan grave sería.

Control

Un control busca reducir probabilidad, impacto o ambos.

Ejemplos

- MFA;
- autorización;
- copias de seguridad;
- validación de entrada;
- monitoreo;
- revisión de código;
- segmentación de privilegios.

Verificación

Después de aplicar un control, debe comprobarse si realmente funciona.

Ejemplo

Si se corrige una vulnerabilidad de autorización, se debe intentar nuevamente acceder al recurso de otro usuario y confirmar que el sistema lo bloquea.

Diez ejemplos rápidos

1. **API sin autenticación** → vulnerabilidad.
2. **Atacante externo** → amenaza.
3. **Consulta no autorizada de datos** → ataque.
4. **Fuga de información** → impacto.
5. **MFA** → control.
6. **Base de datos de clientes** → activo.
7. **Servidor sin energía** → amenaza ambiental.
8. **Contraseña 123456** → vulnerabilidad.
9. **Intento de fuerza bruta** → ataque.
10. **Pérdida económica** → impacto.

Cinco casos aplicados a software

Caso 1 — Sistema académico

Activo: notas. Vulnerabilidad: autorización insuficiente. Amenaza: estudiante curioso o malintencionado. Ataque: cambiar ID en la URL. Impacto: acceso a notas ajenas. Control: autorización por recurso.

Caso 2 — Comercio electrónico

Activo: precios y pedidos. Vulnerabilidad: precio enviado por el cliente y confiado por el servidor. Ataque: manipular el valor antes de enviar la compra. Impacto: pérdida económica. Control: calcular precio en servidor.

Caso 3 — Aplicación empresarial

Activo: documentos internos. Vulnerabilidad: bucket público. Amenaza: tercero externo. Ataque: acceso directo. Impacto: divulgación. Control: permisos privados.

Caso 4 — Credenciales

Activo: cuenta de administrador. Vulnerabilidad: contraseña débil. Ataque: diccionario. Impacto: toma de cuenta. Control: MFA, política de contraseñas y límite de intentos.

Caso 5 — Disponibilidad

Activo: API crítica. Vulnerabilidad: operación costosa sin límites. Ataque: gran número de solicitudes. Impacto: saturación. Control: rate limiting y optimización.

Ejercicio de clasificación

Clasifica cada elemento: activo, amenaza, vulnerabilidad, ataque, impacto, riesgo o control.

1. Clave API embebida en código.
2. Robo de credenciales.
3. Datos de clientes.
4. Usuario interno malintencionado.
5. MFA.
6. Alteración de registros.
7. Riesgo de acceso no autorizado.
8. Intentos automatizados de contraseña.
9. Base de datos pública.
10. Revisión de permisos.

1.2.1 — Tipos de amenazas

1.2.1 Tipos de amenazas

Las amenazas no provienen únicamente de atacantes. Un sistema puede fallar por causas naturales, ambientales o humanas.

Amenazas naturales

Son eventos de origen natural que pueden afectar infraestructura, datos y continuidad.

Ejemplos

- terremotos;
- inundaciones;
- incendios forestales;
- tormentas severas.

Impacto sobre software

Aunque el desarrollador no controle el evento natural, sí debe considerar continuidad, copias de seguridad, redundancia y recuperación.

Amenazas ambientales

Se originan en condiciones del entorno físico o técnico.

Ejemplos

- fallo eléctrico;
- sobretemperatura;
- humedad;
- fallo de almacenamiento;
- deterioro de hardware;
- problemas de climatización.

Ejemplo

Una aplicación está bien programada, pero la base de datos se vuelve inaccesible por fallo del almacenamiento. La disponibilidad queda afectada.

Amenazas humanas accidentales

Ocurren sin intención maliciosa.

Ejemplos

- borrar una tabla por error;
- publicar una clave en un repositorio;
- configurar incorrectamente permisos;
- desplegar una versión equivocada;
- enviar información al destinatario incorrecto.

Relación con desarrollo

Muchos incidentes de software no requieren un atacante sofisticado; una mala configuración puede ser suficiente.

Amenazas humanas intencionales

Existe intención de causar daño, obtener beneficio o acceder sin autorización.

Ejemplos

- fraude;
- sabotaje;
- robo de información;
- abuso de privilegios;
- extorsión;
- espionaje.

Comparación

Tipo	Ejemplo	Efecto típico
Natural	Terremoto	Interrupción o pérdida física
Ambiental	Fallo eléctrico	Caída del servicio
Humana accidental	Secreto publicado	Exposición de credenciales
Humana intencional	Robo de datos	Compromiso de confidencialidad

Caso de análisis

Un desarrollador ejecuta por error un script contra la base de datos de producción y elimina registros.

- Tipo de amenaza: humana accidental.
- Activo: datos de producción.
- Impacto: pérdida de integridad y disponibilidad.
- Controles: backups, permisos mínimos, separación de entornos y confirmación antes de operaciones destructivas.

- # 1.2.2 — Tipos de atacantes
- El objetivo de esta sección no es memorizar etiquetas, sino entender que distintos actores tienen **motivaciones, capacidades y objetivos diferentes**.
- ## Atacante externo
- Persona o grupo sin acceso legítimo previo al sistema.
- **Motivaciones**
- Económica, espionaje, curiosidad, reconocimiento o daño.
- **Escenario**
- Un tercero intenta aprovechar una API expuesta públicamente.
- ## Usuario interno
- Persona que posee acceso legítimo por su rol, pero puede cometer errores o usar sus permisos de forma indebida.
- **Escenario**
- Un empleado descarga información a la que puede acceder, pero la comparte fuera de la organización.
- ## Insider malicioso
- Usuario interno que deliberadamente abusa de su posición.
- **Motivación**
- Venganza, beneficio económico, sabotaje o espionaje.
- ## Ciberdelincuente
- Actor cuya motivación suele ser económica y que participa en fraude, robo de cuentas, extorsión o comercialización de datos.
- ## Script kiddie
- Persona con conocimiento limitado que utiliza herramientas o exploits creados por terceros.
- **Importante**
- Capacidad técnica baja no significa riesgo nulo: una vulnerabilidad pública y fácil de explotar puede ser suficiente.
- ## Hacktivista
- Actor motivado por objetivos ideológicos, sociales o políticos.
- **Objetivos posibles**
- Divulgación, protesta digital, alteración de sitios o interrupción de servicios.
- ## Grupos organizados
- Equipos coordinados con especialización, recursos y continuidad operativa.
- **Ejemplo**
- Campañas de fraude, ransomware o robo de información desarrolladas de forma estructurada.
- ## Actores estatales
- Actores asociados o vinculados a intereses estatales y con recursos elevados.
- **Objetivos típicos**
- Espionaje, inteligencia, influencia o acceso estratégico.

- # Tabla comparativa

Actor	Motivación típica	Conocimiento aproximado	Objetivo frecuente
Script kiddie	Curiosidad/reconocimiento	Bajo	Sistemas vulnerables
Ciberdelincuente	Económica	Medio-alto	Fraude, cuentas, datos
Insider	Variable	Conoce el entorno	Datos o procesos internos
Grupo organizado	Económica/estratégica	Alto	Campañas sostenidas
Actor estatal	Estratégica	Alto	Información sensible

- # Actividad
- Para cada escenario, identifica actor probable, motivación, activo objetivo y nivel aproximado de conocimiento.

1.2.3 — Ataques más comunes

Pregunta de inicio: ¿todos los ataques buscan destruir sistemas? No. Algunos observan, otros suplantan, otros reutilizan información y otros buscan interrumpir servicios.

Este capítulo desarrolla cada ataque con **definición, características, ejemplos, impacto, señales, prevención y relación con desarrollo de software**.

Malware

El **malware** es software o código diseñado para realizar acciones maliciosas o no autorizadas sobre un sistema, dispositivo o información.

Características generales

- puede ejecutarse de forma visible u oculta;
- puede robar, cifrar, alterar o destruir información;
- puede propagarse o depender de la acción del usuario;
- puede afectar confidencialidad, integridad y disponibilidad.

Virus

Un **virus** se adjunta a un archivo o programa legítimo y normalmente necesita que ese archivo se ejecute para activarse o propagarse.

Ejemplo

Un archivo ejecutable modificado infecta otros archivos cuando el usuario lo abre.

Prevención

Actualizaciones, control de software, firmas digitales, antivirus/EDR y restricciones de ejecución.

Gusano

Un **gusano** puede propagarse de forma automática entre sistemas aprovechando redes o vulnerabilidades.

Diferencia con un virus

El virus suele depender de un archivo huésped; el gusano se propaga por sí mismo.

Troyano

Un **troyano** se presenta como software legítimo o útil, pero contiene funcionalidades maliciosas ocultas.

Ejemplo

Una supuesta herramienta gratuita instala un componente que roba credenciales.

Ransomware

El **ransomware** busca impedir el acceso a información o sistemas, generalmente mediante cifrado o bloqueo, para exigir un pago o beneficio.

Impacto

- indisponibilidad;
- pérdida de datos;
- interrupción operativa;
- extorsión;
- daño reputacional.

Controles

Backups probados, segmentación, mínimos privilegios, actualización y respuesta a incidentes.

Spyware

El **spyware** monitorea actividades del usuario o del sistema para recopilar información sin autorización.

Información buscada

Credenciales, navegación, documentos, capturas de pantalla o información personal.

Desarrollo seguro frente a malware

- validar archivos cargados;
- limitar ejecución de contenido;
- verificar dependencias;
- firmar software;
- reducir privilegios;
- no confiar en extensiones de archivo solamente.

Snooping

El **Snooping**, también conocido como *eavesdropping* o espiamiento, consiste en observar, escuchar o monitorear de forma no autorizada información, comunicaciones o actividades con el objetivo de obtener datos sensibles.

A menudo se caracteriza por ser **pasivo**: el atacante intenta obtener información sin modificarla ni interrumpir el servicio, lo que puede hacer más difícil su detección.

Formas comunes de snooping

Tipo	Descripción	Ejemplo
Observación de comunicaciones	Escuchar o revisar comunicaciones sin autorización	Leer mensajes expuestos
Observación física	Observar pantalla o teclado	Shoulder surfing
Monitoreo mediante software	Registrar actividad de un dispositivo	Spyware o keylogger

Tipo	Descripción	Ejemplo
Captura de tráfico	Observar información que circula por una red	Sniffing

¿Qué información busca?

- usuarios y contraseñas;
- datos bancarios;
- mensajes;
- documentos internos;
- datos personales;
- tokens o sesiones expuestas.

Ejemplo aplicado a software

Una aplicación envía credenciales en texto claro mediante HTTP. Un observador con acceso al segmento de red puede capturar esas credenciales sin alterar la comunicación.

Medidas de prevención

1. Cifrar comunicaciones con HTTPS/TLS.
2. Evitar redes inalámbricas inseguras.
3. Proteger físicamente pantallas y dispositivos.
4. Utilizar controles antimalware.
5. No registrar secretos innecesarios en logs.
6. Diseñar aplicaciones para no transmitir datos sensibles sin protección.

Sniffing

El **sniffing** es la captura y análisis de tráfico de red para observar paquetes o comunicaciones que circulan entre sistemas.

Sniffing legítimo vs malicioso

El análisis de tráfico puede ser legítimo para diagnóstico, monitoreo o administración. Se convierte en un ataque cuando se realiza sin autorización para obtener información o apoyar otras acciones maliciosas.

HTTP vs HTTPS

flowchart LR

A["Usuario"] -- "HTTP: datos potencialmente legibles" --> B["Servidor"]

C["Usuario"] -- "HTTPS/TLS: canal cifrado" --> D["Servidor"]

Información observable en tráfico inseguro

- credenciales;
- parámetros;
- cookies;
- contenido de formularios;
- datos personales;
- tokens si viajan sin protección.

Prevención

HTTPS/TLS, redes seguras, VPN en escenarios apropiados y eliminación de protocolos inseguros.

Snooping vs Sniffing

Snooping es un concepto más amplio de espionaje u observación no autorizada. **Sniffing** es una técnica específica de captura de tráfico de red.

Aspecto	**Snooping**	**Sniffing**
Alcance	Amplio	Tráfico de red
Ejemplo	Observar pantalla	Capturar paquetes
Naturaleza	Espionaje	Técnica de captura

Spoofing

El **spoofing** consiste en falsificar o suplantar una identidad, dirección, origen o componente para que otro sistema confíe en información falsa.

Ejemplos conceptuales

- correo con remitente falsificado;

- suplantación de dirección IP;
- sitio web que imita uno legítimo;
- DNS alterado para dirigir al usuario a un destino falso.

Riesgo para aplicaciones

Una aplicación que confía únicamente en datos fácilmente falsificables, como un encabezado HTTP no validado, puede tomar decisiones incorrectas de seguridad.

Prevención

Autenticación fuerte, validación de origen, TLS, certificados y diseño que no dependa de identificadores falsificables como único control.

Masquerading

El **masquerading** ocurre cuando un atacante se presenta como un usuario, sistema o entidad legítima, normalmente utilizando credenciales, sesiones o identidades comprometidas.

Ejemplo

Un atacante roba una sesión válida y actúa dentro de la aplicación como si fuera el usuario legítimo.

Spoofing vs Masquerading

Aspecto	**Spoofing**	**Masquerading**
Idea principal	Falsificar origen o identidad	Actuar como entidad legítima
Ejemplo	Remitente falso	Usar sesión robada
Objetivo	Engañar confianza	Obtener privilegios o acceso

Replay Attack

Un **replay attack** ocurre cuando una comunicación o solicitud válida es capturada y posteriormente reenviada para intentar que el sistema la acepte nuevamente.

Flujo

flowchart LR

A["Solicitud legítima"] --> B["Captura"] --> C["Reenvío"] --> D["Sistema la acepta otra vez"]

Ejemplo

Una solicitud de aprobación contiene un token reutilizable. Si el servidor no comprueba unicidad, vigencia o contexto, la misma solicitud podría reutilizarse.

Controles introductorios

- tokens de un solo uso;
- timestamps;
- nonces;
- expiración;
- asociación del token con contexto o sesión;
- protección criptográfica de mensajes.

DoS — Denial of Service

Un ataque de **denegación de servicio** busca impedir o degradar el acceso de usuarios legítimos a un servicio, sistema o recurso.

Formas conceptuales

- saturación de solicitudes;
- agotamiento de CPU o memoria;
- consumo excesivo de conexiones;
- explotación de una operación costosa;
- abuso de recursos limitados.

Ejemplo aplicado a software

Una API permite generar un reporte extremadamente costoso sin límites. Un usuario realiza cientos de solicitudes simultáneas y agota los recursos del servidor.

Prevención desde desarrollo

- rate limiting;
- límites de tamaño;
- timeouts;
- colas;

- caché;
- paginación;
- operaciones asincrónicas;
- manejo correcto de conexiones.

Ataques a contraseñas

Los ataques a contraseñas intentan obtener acceso mediante credenciales débiles, reutilizadas, filtradas o predecibles.

Contraseñas débiles

Contraseñas cortas, comunes o relacionadas con el usuario.

Ejemplos: 123456, password, admin123.

Ataque de diccionario

Prueba listas de palabras o contraseñas comunes en lugar de todas las combinaciones posibles.

Fuerza bruta

Prueba sistemáticamente múltiples combinaciones hasta encontrar una válida.

Reutilización de contraseñas

Si una persona usa la misma contraseña en varios servicios, una filtración puede comprometer otras cuentas.

Credenciales por defecto

Equipos o aplicaciones pueden venir con cuentas predeterminadas que deben cambiarse.

Credential stuffing

Utiliza combinaciones usuario/contraseña obtenidas de filtraciones anteriores para intentar acceder a otros servicios.

¿Cómo debe defenderse una aplicación?

- MFA;
- límite de intentos;
- bloqueo temporal;

- detección de comportamiento anómalo;
- mensajes de error genéricos;
- almacenamiento seguro de contraseñas;
- impedir contraseñas conocidas como comprometidas cuando corresponda.

Ingeniería social

La **ingeniería social** manipula a personas para obtener información, acceso o acciones que beneficien al atacante.

Phishing

Mensajes fraudulentos, normalmente por correo o web, que intentan engañar al usuario para entregar credenciales o realizar acciones.

Ejemplo

Correo que imita a la universidad y solicita “verificar la cuenta” mediante un enlace falso.

Smishing

Phishing realizado mediante SMS o mensajería móvil.

Vishing

Engaño mediante llamadas de voz.

Pretexting

El atacante construye una historia o contexto falso para obtener información o cooperación.

Ejemplo

Una persona se presenta como técnico de soporte y solicita un código MFA.

Baiting

Ofrece algo atractivo para provocar una acción insegura.

Ejemplo

Un dispositivo USB etiquetado “nómina” dejado intencionalmente para que alguien lo conecte.

Relación con desarrollo de software

Una aplicación segura debe reducir el daño cuando el usuario se equivoca:

- MFA;
- alertas de inicio de sesión;
- confirmación de operaciones críticas;
- mecanismos anti-phishing;
- gestión segura de sesiones;
- recuperación de cuenta robusta.

Caso integrador

Un estudiante recibe un correo falso, introduce su contraseña en una página clonada y el atacante usa esas credenciales para entrar al sistema académico.

- **Ingeniería social:** phishing.
- **Masquerading:** el atacante actúa como el estudiante.
- **Activo:** cuenta y datos académicos.
- **Control preventivo:** MFA y educación.
- **Control detectivo:** alerta por inicio desde ubicación anómala.
- **Impacto:** acceso no autorizado y posible modificación de información.

Actividad de análisis

Para cada escenario, identifica: tipo de ataque, propiedad CIA afectada, activo, impacto y control principal.

1. Captura de tráfico HTTP.
2. Reutilización de una solicitud válida.
3. Saturación de una API.
4. Uso de contraseña filtrada.
5. Llamada de un supuesto técnico solicitando MFA.

1.3.1 — Tipos de controles

1.3.1 Tipos de controles

Un **control de seguridad** es una medida destinada a reducir la probabilidad de ocurrencia de un evento, detectar que está ocurriendo o disminuir sus consecuencias.

Controles preventivos

Intentan impedir que el incidente ocurra.

Ejemplos

- MFA;
- autorización;
- validación de entrada;
- permisos mínimos;
- actualizaciones;
- políticas;
- segmentación;
- cifrado.

Caso

Una API valida que el usuario sea propietario del recurso antes de entregar información.

Controles detectivos

Buscan identificar eventos, anomalías o incidentes cuando ocurren o después de que han ocurrido.

Ejemplos

- logs;
- alertas;
- monitoreo;
- detección de múltiples intentos fallidos;
- revisión de eventos;
- auditoría.

Caso

El sistema genera una alerta al detectar 50 intentos fallidos desde una misma dirección.

Controles correctivos

Buscan reducir consecuencias y recuperar el estado normal después de un problema.

Ejemplos

- restaurar backup;
- cambiar credenciales;
- corregir vulnerabilidad;
- aislar componente;
- reinstalar servicio;
- aplicar parche.

Un mismo riesgo necesita varios controles

Ejemplo: toma de cuenta

Riesgo	Preventivo	Detectivo	Correctivo
Acceso no autorizado	MFA + límite de intentos	Alertas de login anómalo	Revocar sesiones + cambio de credenciales
Modificación de datos	Autorización	Logs de cambios	Restauración o reversión
Caída de servicio	Redundancia y límites	Monitoreo	Failover y recuperación

Controles técnicos, administrativos y físicos

Otra clasificación útil considera la naturaleza del control.

Técnicos

Software o mecanismos tecnológicos: MFA, WAF, cifrado, permisos.

Administrativos

Políticas, procesos, capacitación, segregación de funciones.

Físicos

Control de acceso a instalaciones, cerraduras, CCTV, protección de equipos.

Defensa en profundidad

No conviene depender de un único control.

flowchart LR

A["Usuario"] --> B["MFA"] --> C["Autorización"] --> D["Validación"] --> E["Logs y alertas"] --> F["Backup / recuperación"]

Error frecuente

Pensar que un control detectivo “previene” el ataque. Un log puede ayudar a detectar o investigar, pero por sí solo no impide el intento.

Actividad

Para cada riesgo, propone un control preventivo, uno detectivo y uno correctivo:

1. Robo de cuenta.
2. Modificación de notas.
3. Eliminación accidental de datos.
4. API saturada.
5. Secreto publicado en repositorio.

1.3.2 — Introducción a normas y estándares

Pregunta de inicio: ¿por qué una organización necesita un marco si ya tiene antivirus, firewall y desarrolladores?

Las herramientas aisladas no garantizan una estrategia coherente. Los marcos y estándares proporcionan estructura, lenguaje común, objetivos y prácticas para gestionar la seguridad de forma sistemática.

¿Qué es un marco de seguridad?

Un **marco** organiza prácticas, actividades y resultados deseados para ayudar a gestionar riesgos.

Ejemplo

NIST Cybersecurity Framework ayuda a estructurar actividades de gobierno, identificación, protección, detección, respuesta y recuperación.

¿Qué es una norma?

Una **norma** establece requisitos o lineamientos formales que pueden ser utilizados para gestionar, evaluar o certificar determinadas prácticas.

Ejemplo

ISO/IEC 27001 establece requisitos para un Sistema de Gestión de Seguridad de la Información (SGSI).

Familia ISO/IEC 27000

La familia **ISO/IEC 27000** agrupa normas relacionadas con gestión de seguridad de la información.

Papel general

- vocabulario;
- requisitos de gestión;
- controles;
- gestión de riesgos;
- implementación y mejora.

ISO/IEC 27001

ISO/IEC 27001 define requisitos para establecer, implementar, mantener y mejorar continuamente un **Sistema de Gestión de Seguridad de la Información (SGSI)**.

Qué busca

Que la organización gestione la seguridad de forma sistemática, basada en riesgos y mejora continua.

Relación con desarrollo de software

Aunque no es una norma de programación, puede influir en controles de acceso, gestión de cambios, desarrollo seguro, incidentes, activos y proveedores.

NIST Cybersecurity Framework 2.0

NIST CSF 2.0 organiza resultados de ciberseguridad alrededor de seis funciones:

1. **Govern** — Gobernar.
2. **Identify** — Identificar.
3. **Protect** — Proteger.
4. **Detect** — Detectar.
5. **Respond** — Responder.
6. **Recover** — Recuperar.

flowchart LR

G["Govern"] --> I["Identify"] --> P["Protect"] --> D["Detect"] --> R["Respond"] --> RC["Recover"]

RC --> G

Govern

Define contexto, roles, políticas, supervisión y estrategia de gestión del riesgo.

Identify

Permite entender activos, dependencias, contexto y riesgos.

Ejemplo

Inventariar aplicaciones, APIs, repositorios, datos y terceros.

Protect

Aplica salvaguardas para reducir riesgos.

Ejemplos

MFA, autorización, cifrado, seguridad de desarrollo y capacitación.

Detect

Permite descubrir eventos y comportamientos anómalos.

Ejemplos

Logs, alertas, monitoreo de accesos y detección de actividad inusual.

Respond

Define acciones ante incidentes detectados.

Ejemplos

Contención, comunicación, investigación y mitigación.

Recover

Busca restaurar capacidades y servicios después de un incidente.

Ejemplos

Restaurar backups, recuperar servicios y mejorar controles.

ISO/IEC 27001 vs NIST CSF

Aspecto	**ISO/IEC 27001**	**NIST CSF 2.0**
Naturaleza	Norma con requisitos para SGSI	Marco de resultados y gestión
Enfoque	Sistema de gestión basado en riesgos	Organización de resultados de ciberseguridad
Uso en clase	Entender gestión formal	Entender ciclo de gestión y funciones

Caso aplicado a una aplicación

Una empresa lanza una aplicación de comercio electrónico.

- **Govern:** define responsables y políticas.
- **Identify:** identifica datos de clientes, API de pagos y dependencias.
- **Protect:** aplica MFA a administradores y controles de acceso.
- **Detect:** monitorea intentos de acceso anómalos.
- **Respond:** bloquea sesiones comprometidas y comunica incidente.
- **Recover:** restaura servicios y revisa controles.

Idea de cierre

Los marcos no sustituyen a los controles técnicos. Ayudan a decidir **qué proteger, cómo organizar responsabilidades, cómo detectar problemas y cómo recuperarse**.

4. Conceptos y principios

4.1 Conceptos

4.1.1 Necesidad de la seguridad de la información

Las organizaciones recopilan, procesan, almacenan, transmiten y eliminan información; determinados datos y los recursos relacionados constituyen activos. La información también puede pertenecer a clientes u otras organizaciones. La divulgación, pérdida de integridad o falta de disponibilidad puede producir daños. Las amenazas incluyen errores humanos, fraudes, robos, espionaje, sabotaje, incendios, inundaciones, ataques informáticos y denegación de servicio. La combinación de probabilidad y consecuencia caracteriza el riesgo, que debe gestionarse y, si resulta inaceptable, tratarse.

4.1.2 Información

La información es un activo esencial para las actividades de una organización y debe protegerse adecuadamente, sea propia o confiada por terceros. Puede encontrarse en soportes digitales, ópticos, papel o conocimientos, y transmitirse por medios físicos, electrónicos o verbales. Su protección es necesaria independientemente de la forma o el medio.

4.1.3 Seguridad de la información

La seguridad de la información asegura la confidencialidad, disponibilidad e integridad mediante la aplicación y gestión de controles adecuados frente a diversas amenazas, favoreciendo la continuidad y reduciendo las consecuencias de incidentes.

4.1.4 Riesgos en constante cambio

Los riesgos no son estáticos. Cambian con la organización, su entorno y las capacidades e intereses de los atacantes. Por ello, las organizaciones deben supervisar la eficacia de sus controles, reevaluar periódicamente sus riesgos y mantener sus planes de tratamiento.

4.1.5 Plan de tratamiento de riesgos

Los controles de seguridad se determinan mediante la evaluación y el tratamiento de riesgos. Se clasifican en organizacionales, relativos a personas, físicos y tecnológicos; asimismo, pueden ser preventivos, detectivos o correctivos. Una combinación adecuada permite resistir fallos de control.

4.1.6 Finalidad de un SGSI

Un SGSI es un sistema de gestión destinado a preservar la seguridad de la información mediante políticas, procesos, personas, elementos físicos, tecnología y recursos. ISO/IEC 27001 especifica sus requisitos y exige un marco de gestión para evaluar y tratar riesgos. El SGSI ayuda a atender requisitos de las partes interesadas, cumplir objetivos y obligaciones legales o contractuales y proteger activos.

4.1.7 Importancia de un SGSI

Adoptar un SGSI es una decisión estratégica que debe integrarse en las necesidades, actividades, dimensiones y estructura de la organización. Su diseño y funcionamiento responden a los requisitos de las partes interesadas y facilitan una gestión coherente de los riesgos y de la seguridad.

4.1.8 Enfoque basado en procesos

La salida de un proceso puede convertirse directamente en entrada de otro. La organización realiza estas transformaciones en condiciones planificadas y controladas.

4.1.9 Alcance

El alcance de un SGSI comprende todo aquello que gestiona y toda la información protegida o que debe protegerse mediante sus controles.

4.2 Principios

- **4.2.1 Establecer, implementar, mantener y mejorar el SGSI:** definir qué información queda en el alcance, determinar requisitos, evaluar y tratar riesgos, definir controles y revisar periódicamente su eficacia.
- **4.2.2 Implementación satisfactoria del SGSI:** conciencia de seguridad, asignación de responsabilidades, compromiso de dirección, gestión del riesgo, integración en procesos, detección y respuesta a incidentes, enfoque integral y reevaluación continua.
- **4.2.3 Determinación de requisitos de seguridad de la información:** considerar estrategia y objetivos, ubicaciones, activos y su valor, necesidades de procesamiento y comunicación, así como requisitos legales, reglamentarios y contractuales.

- **4.2.4 Integración en procesos de negocio:** los controles de seguridad de la información deben integrarse de manera fluida en los procesos de negocio de la organización.